

DSA Coursework: Simple Step-by-Step Guide

Purpose

This guide explains exactly what to do for **Lab 1: Sorting, Complexity** in the repository ¹. The lab has two required parts: analysing the complexity of three sorting algorithms and improving quicksort through controlled experiments. ¹

Important academic rule: This lab is part of the course examination. Discussing ideas is allowed, but your code, measurements, and written answers must be your own work and the work of your lab partners. Do not copy code from or show code to another group. ¹

1. Understand the files

First, download or clone the repository. You need a Java compiler such as `javac`, or a Java IDE such as Visual Studio Code, NetBeans, IntelliJ, or Eclipse. ¹

The important files are:

File	What it is for
<code>Insertion.java</code>	Insertion-sort implementation.
<code>Merge.java</code>	Top-down merge-sort implementation.
<code>Quick.java</code>	Quicksort implementation that initially uses the first element as pivot. This is the file you must improve.
<code>Bench.java</code>	Tests whether the algorithms sort correctly and measures their running time.
<code>answers.txt</code>	The file where you must write your answers, group information, explanations, and declarations.
<code>README.md</code>	The official lab instructions.
<code>Lab-1-README.pdf</code>	Another copy of the lab instructions.

2. Make a safe working copy

Create a folder for your group and copy all repository files into it. Keep an untouched copy of the original repository so you can restore the starter code if necessary.

If you are using a terminal, run:

Bash

```
git clone https://github.com/MubiruEltonFelix1/dsa-coursework-year-two.git
cd dsa-coursework-year-two
cp -r . ../dsa-coursework-year-two-backup
```

Do not edit `answers.txt` immediately. First, read the whole file so you know every question that must be completed, including the appendix.

3. Compile and run the original program

Before changing anything, compile all Java files:

Bash

```
javac *.java
```

Then run the benchmark and correctness tests:

Bash

```
java Bench
```

Save the original output in a file. The README requires an `output.txt` file for the final submission, so you can begin recording results with:

Bash

```
java Bench | tee output-original.txt
```

The benchmark tests arrays of different sizes and three input conditions:

Benchmark label	Meaning
Random	Completely random input arrays.
95% sorted	Arrays in which 95% of the elements are in the correct order and the remainder are random.

Sorted

Already sorted arrays.

For each algorithm and input condition, the report gives **Total** time and **per item** time. The per-item value is total time divided by the input size. ¹

Check that the original benchmark completes without a sorting error. If it reports **Test failed!**, stop and investigate before making changes.

4. Complete Task 1: determine complexity from the timing tables

Use the timing reports produced by `Bench.java` for `Insertion.java`, `Quick.java`, and `Merge.java`.

For each algorithm, determine the complexity separately for:

1. Random input.
2. 95% sorted input.
3. Already sorted input.

Do not decide from the source code. The instructions specifically require you to use the timing data only. ¹

Use the **per item** column as your main evidence:

If total runtime behaves like	The per-item time usually behaves like
n^2	It grows approximately in proportion to n^2 .
$n \log n$	It grows approximately like $\log n$.
n	It stays approximately constant as n increases.

For accuracy, compare several increasing sample sizes rather than one row. Also remember that the complexity describes how runtime changes with input size; it is not the same as which algorithm is fastest. ¹

In `answers.txt`, select one answer for every row under the Task 1 section. Do not leave any of the nine algorithm/input combinations blank.

5. Understand the three required quicksort improvements

Open `Quick.java` and find the three `TODO` sections. Implement the following improvements **one at a time**.

Improvement A: Shuffle once before sorting

When `shuffleFirst` is `true`, call the existing `shuffle(int[] array)` method once at the beginning of `sort(int[] a)`, before the recursive quicksort starts. Do not shuffle inside recursive calls. ¹

The purpose is to reduce the chance that an input array produces badly unbalanced partitions.

Improvement B: Use median-of-three pivot selection

When `useMedianOfThree` is `true`, examine the first, middle, and last elements of the current subarray `a[lo..hi]`. Use the existing `medianOfThree` method to find the index of the median value, then use the existing `exchange` method to swap that element with `a[lo]`. The existing partition code can then use `a[lo]` as the pivot. ¹

Be careful to calculate the middle index from the current subarray, not from the whole array.

Improvement C: Use insertion sort for small subarrays

In the recursive `sort(a, lo, hi)` method, check whether the current subarray is smaller than the configured `insertionSortCutoff`. If it is, call:

Java

```
Insertion.sort(a, lo, hi);
```

Then return so quicksort does not continue recursing on that subarray. You do not need to implement insertion sort yourself. ¹

Use a sensible boundary consistently. The cutoff must be tested experimentally, not guessed.

6. Test after every code change

After implementing only one improvement, compile and run the benchmark again:

Bash

```
javac *.java
java Bench
```

Confirm both of the following:

Check	What must happen
Correctness	<code>Bench.java</code> must not report a sorting failure.
Performance	Record whether the timing improves for random, 95% sorted, and sorted inputs.

If a test fails, use the failing input printed by `Bench.java`. You may temporarily add a small `main` method to `Quick.java` that sorts that exact input, then step through the algorithm in your IDE. Remove debugging code before submission. ¹

Do not implement all three improvements at once at the beginning. The required experiment is to make and measure them separately so you can explain what each one changes. ¹

7. Benchmark every useful combination

`Quick` is configured through this constructor:

Java

```
public Quick(boolean shuffleFirst, boolean useMedianOfThree, int insertionSortC
```

The three arguments mean:

Constructor argument	<code>true</code> or positive value means
<code>shuffleFirst</code>	Shuffle the array before quicksort begins.
<code>useMedianOfThree</code>	Select the pivot using median-of-three.
<code>insertionSortCutoff</code>	Use insertion sort below this subarray-size cutoff. A value of <code>0</code> disables the cutoff.

In `Bench.java`, keep the original no-improvement benchmark:

Java

```
executionTimeReport("Quick.java: quicksort", new Quick(false, false, 0)::sort);
```

Also add clearly labelled benchmark lines for the versions you test. For example:

Java

```
executionTimeReport("Quick: shuffle only", new Quick(true, false, 0)::sort);
executionTimeReport("Quick: median-of-three only", new Quick(false, true, 0)::s
executionTimeReport("Quick: insertion cutoff 42", new Quick(false, false, 42)::
executionTimeReport("Quick: all improvements, cutoff 42", new Quick(true, true,
```

You may add more lines to test other combinations and cutoff values. Change the label every time so you can identify the results in `output.txt` . 1

8. Find the best insertion-sort cutoff systematically

Do not claim that a cutoff is best after testing only one value. Use this process:

1. Choose an initial cutoff, such as the example value `42` in the README.
2. Test the same quicksort configuration with several nearby values, such as 10, 20, 30, 40, 50, and 60.
3. Keep the benchmark environment as consistent as possible between runs.
4. Compare the same input sizes and the same three degrees of sortedness each time.
5. Repeat a promising test if the results are close or noisy.
6. Select the cutoff with the best overall performance, not merely the best single timing row.
7. Test values above and below your selected value. Performance should get worse in both directions, or at least show that your choice is a reasonable local optimum.
8. Record the tested values and the evidence in your notes before writing `answers.txt` .

A cutoff does not need to be exact because results depend on the computer. Rounding to the nearest 10 is considered reasonable by the README. 1

9. Decide which combination is best

Compare the no-improvement version with versions using:

1. Shuffling only.
2. Median-of-three only.
3. Insertion sort for small subarrays only.

4. Shuffling plus median-of-three.
5. Shuffling plus insertion cutoff.
6. Median-of-three plus insertion cutoff.
7. All three improvements together.

You do not have to use every possible combination if your experiments clearly establish the result, but you must make enough comparisons to justify your conclusion. The instructions explicitly warn that adding all three improvements may not automatically be best. ¹

Choose the version that gives the best practical performance while still passing all correctness tests. Record its constructor settings exactly, including the cutoff value.

10. Complete Task 2 in `answers.txt`

Fill in every part of the Task 2 section using your own measurements and explanations.

Question	What to write
Which improvements affect complexity?	State separately what happened with shuffling, median-of-three, and insertion cutoff. Explain the effect for the relevant input types. Base this on your timing tables.
What cutoff works best?	Give the selected cutoff and briefly describe the values tested, the comparison method, and why nearby values were worse or less effective.
Which combination is best?	Name the exact combination and constructor settings, then summarize the benchmark evidence.

Do not write unsupported claims such as “this is faster” without identifying the input type or timing evidence.

11. Complete the appendix in `answers.txt`

The starter `answers.txt` also asks for information that must not be skipped:

Appendix section	Required action
Group members	Enter each student number, registration number, and name.

Hours spent	Enter the approximate hours spent by each group member.
Known bugs or limitations	State any remaining issue honestly. Write <code>None known</code> only after testing carefully.
Collaboration	Describe collaboration with other students, if any, and list resources used, including web resources.
Serious problems	Describe serious problems encountered, or state that none occurred.
Other comments	Add useful feedback about what you learned or any relevant comments.

Never leave placeholder text such as `[...]`, `[..hours..]`, or `[..group member..]` in the final file.

12. Produce the final benchmark output

Run the final version of `Bench.java` after you have selected the improvements. Make sure the output includes:

1. The three sorting algorithms: insertion sort, merge sort, and the original/no-improvement quicksort benchmark.
2. The improved quicksort configuration you chose.
3. The correctness tests performed by `Bench.java`.
4. The timing reports for random, 95% sorted, and sorted inputs.

Save the output as `output.txt` :

Bash

```
javac *.java
java Bench > output.txt
```

Open `output.txt` and check that it is not empty, the labels identify each algorithm/configuration, and no failure message appears.

13. Perform the final quality check

Before creating the ZIP file, use this checklist:

Check	Complete?
<code>Quick.java</code> contains the required improvements you selected.	☐
<code>Quick.java</code> still compiles.	☐
All tested versions sort correctly.	☐
The original no-improvement benchmark remains in <code>Bench.java</code> .	☐
<code>answers.txt</code> contains all nine Task 1 complexity answers.	☐
<code>answers.txt</code> explains the effect of all three improvements.	☐
<code>answers.txt</code> gives a tested cutoff and explains how it was found.	☐
<code>answers.txt</code> identifies the best combination.	☐
Group and appendix information is complete.	☐
<code>output.txt</code> contains the final benchmark output.	☐
No temporary debugging code or irrelevant files are included.	☐
The final files compile and run from a clean folder.	☐

A useful final clean-folder test is:

Bash

```
mkdir final-check
cp Quick.java Bench.java Insertion.java Merge.java answers.txt output.txt final
cd final-check
javac *.java
java Bench
```

If the clean-folder test fails, fix the problem before submitting.

14. Create the required ZIP submission

The required submission is a ZIP file named after your group, for example `alpha.zip`. ¹ It must contain:

Plain Text

```
Quick.java  
output.txt  
answers.txt
```

Create it from the folder containing the final files:

Bash

```
zip alpha.zip Quick.java output.txt answers.txt  
unzip -l alpha.zip
```

Replace `alpha` with your actual group name. Use `unzip -l` to verify that the ZIP contains the required files and no accidental backup folders or unrelated files.

15. Optional tasks

The following are optional and should only be attempted after all required work is complete:

- Choose a random pivot directly and compare it with shuffling first.
- Optimise merge sort, including checking whether a subarray is already ordered and possibly using insertion sort for small subarrays.
- Implement another sorting algorithm, such as dual-pivot quicksort, American flag sort, run-based mergesort, or Timsort.
- Implement sorting algorithms in a functional language.
- Create a sorting function that dynamically chooses an algorithm based on the input. ¹

Optional work does not replace the required `Quick.java`, `output.txt`, and completed `answers.txt` submission.

Final submission summary

When finished, submit one ZIP file named with your group name. The ZIP must contain the improved `Quick.java`, the final `output.txt` produced by `Bench.java`, and the fully completed `answers.txt`. ¹

References

- [1] Repository README: Lab 1: Sorting, Complexity
- [2] Repository Quick.java
- [3] Repository Bench.java
- [4] Repository answers.txt